INFORME DE SEGURIDAD Y CALIDAD

EN EL DESARROLLO DE SOFTWARE

API de Alumnos — microservicio Spring Boot (laboratorio calidad 15-all)

ISY1102 - Seguridad y Calidad en el Desarrollo de Software

Sección: ISY1102

Integrantes

Alison Aranda

Macarena Muñoz

Camilo Romero

Carrera: Ingeniería en Informática - Desarrollo de software

Docente: Mauricio Velásquez

Fecha: 16 de octubre de 2026

Documento de referencia: README/OpenAPI del microservicio Alumnos (15-all) y Informe EP1 (plan heredado y adaptado).

Índice

1. Introducción

2. Criterios de calidad, seguridad y cumplimiento

3. Plan de pruebas

4. Diseño de casos de prueba

5. Análisis estático del código fuente

6. Ejecución de pruebas

7. Análisis ético y normativo de los resultados

8. Propuesta de mejora

9. Conclusiones

10. Anexos

1. Introducción

1.1 Contexto del software a evaluar

El software bajo prueba es la **API de Alumnos**, microservicio de ejemplo del curso construido con **Spring Boot 3 + Maven + SQLite**, que expone un CRUD de alumnos (/api/v1/alumnos), documentación **OpenAPI** (Swagger UI / Redoc) y una página home. Sigue arquitectura limpia (dominio / aplicación / infraestructura).

El proyecto 15-all concentra **todas las tecnologías de calidad, seguridad y rendimiento** aplicadas en verde (Spotless, Checkstyle, PMD, SpotBugs, JUnit/Mockito, PIT, JaCoCo, SonarQube, Snyk, Cucumber, Docker/Trivy, OWASP ZAP, Gatling y JMeter). Los módulos 01–14 incluyen además variantes **con-error** / **sin-error** para contrastar hallazgos.

1.2 Arquitectura y módulos críticos

Capa	Contenido	Riesgo
Dominio	Modelo Alumno, excepciones de negocio	Reglas (RUT único, validaciones)
Aplicación	AlumnoUseCase (CRUD)	Lógica de negocio incorrecta
Infraestructura REST	Controllers, DTOs, manejo de errores	Exposición HTTP, códigos 4xx/5xx
Persistencia JPA/SQLite	Gateway + mapper	Integridad de datos
Seguridad transversal	Cabeceras (SecurityHeadersFilter), dependencias, imagen Docker	XSS/clickjacking, CVE, supply chain
Pipeline CI	GitHub Actions de calidad	Gate de calidad/seguridad

Endpoints críticos:

Método	Ruta	Descripción
GET	/api/v1/alumnos	Listar
GET	/api/v1/alumnos/{id}	Obtener
POST	/api/v1/alumnos	Crear
PUT	/api/v1/alumnos/{id}	Actualizar
DELETE	/api/v1/alumnos/{id}	Eliminar

1.3 Propósito del plan de pruebas

Asegurar la **calidad, seguridad y cumplimiento** del microservicio antes de considerarlo apto como base productiva/educativa. La EP01 definió el **diseño** del plan (caso Atlas); esta EP3 **ejecuta** el ciclo sobre el software de calidad del curso (15-all), aplicando análisis estático, pruebas unitarias/integración, DAST, SCA, contenedor y carga, con evidencia y criticidad.

1.4 Alcance

In scope: CRUD alumnos; calidad de código (01–04); unitarias, mutación y cobertura (05–07); SonarQube (08); Snyk (09); Cucumber (10); Docker/Trivy (11); ZAP (12); Gatling/JMeter (13–14); pipeline GitHub Actions.

Out of scope: UI web completa de negocio, autenticación JWT/RBAC de Atlas, módulos no presentes en este microservicio (facturación, etc.).

1.5 Objetivos

1. Verificar el CRUD y las reglas de negocio (RUT único, códigos HTTP correctos).

2. Validar umbrales de calidad: Spotless/Checkstyle/PMD/SpotBugs en verde; JaCoCo 100 %; PIT ≥ 80 %.

3. Confirmar Quality Gate SonarQube y ausencia de vulnerabilidades críticas en Snyk/Trivy.

4. Ejecutar DAST (ZAP) con cabeceras de seguridad en FAIL=0.

5. Validar rendimiento (Gatling/JMeter) sin errores y latencia bajo SLA.

6. Documentar hallazgos con evidencia, criticidad y remediación.

1.6 Estructura del informe

Criterios (2) → plan (3) → casos (4) → análisis estático (5) → ejecución (6) → ética/normativa (7) → mejoras (8) → conclusiones (9) → anexos (10).

1.7 Ajustes respecto de la EP01

La EP01 (09-09-2026) diseñó el plan sobre **Atlas**. Para la EP3 el docente suministra el laboratorio calidad - EJEMPLO; el grupo **hereda** la estructura del plan (estrategia, tipos,

criterios, evidencia) y la **aplica al microservicio Alumnos `15-all`**, que permite ejecutar de punta a punta las herramientas de calidad/seguridad.

#	EP01 / feedback	Ajuste EP3	Sección
1	Plan diseñado sin ejecución documentada	Se ejecuta el ciclo completo sobre 15-all	5–6
2	Caso Atlas (React/Node)	SUT de ejecución: API Alumnos Spring Boot (laboratorio curso)	1
3	Justificación de tipos de prueba	Tabla origen/impacto con herramientas 01–14	3.2
4	Sin SAST formal	Spotless, Checkstyle, PMD, SpotBugs, Sonar, Snyk	5
5	Sin incidentes clasificados	Escala + contraste con módulos con-error	6.3
6	Ética solo en criterios	Análisis ético-normativo de resultados	7
7	Sin remediación por hallazgo	Propuestas + lecciones de con-error	8

2. Criterios de calidad, seguridad y cumplimiento

2.1 Calidad (ISO/IEC 25010)

Atributo	Criterio en Alumnos
Funcionalidad	CRUD correcto; RUT duplicado → 409; no encontrado → 404
Confiabilidad	Tests JUnit + Cucumber en verde; sin 5xx en caminos válidos
Mantenibilidad	Spotless, Checkstyle, PMD, SpotBugs sin violaciones bloqueantes
Cobertura	JaCoCo líneas = 100 % (umbral del proyecto)
Fuerza de pruebas	PIT mutación ≥ 80 %
Rendimiento	Gatling: 0 fallos; media < 2000 ms · JMeter: 0 errores

2.2 Seguridad

Control	Criterio
SCA (dependencias)	Snyk sin vulnerabilidades critical (umbral del proyecto)
Contenedor	Trivy sin CRITICAL/HIGH en paquetes OS de la imagen
DAST	ZAP baseline: FAIL=0 en cabeceras (X-Content-Type-Options, X-Frame-Options, CSP, etc.)
Código	SonarQube sin bugs/vulnerabilidades bloqueantes; Quality Gate OK
Runtime	Usuario no-root en imagen Docker

2.3 Cumplimiento normativo

Marco	Aplicación
Ley 19.628 / 21.719	Datos de alumnos (RUT, email, nombre) tratados en ambiente QA; no exponer datos reales
Ley 21.663	Controles de ciberseguridad: SCA, DAST, hardening de imagen y cabeceras
ISO/IEC 25010 / 27001 / 29119	Modelo de calidad, controles y proceso de pruebas

OWASP	Cabeceras anti-XSS/clickjacking; gestión de dependencias

3. Plan de pruebas

3.1 Estrategia

• **Enfoque mixto:** automatizado (Maven, CI, ZAP, Gatling/JMeter) + revisión manual de reportes y contraste con-error vs sin-error.

• **Etapas:** (1) análisis README/OpenAPI → (2) diseño de casos → (3) mvn verify / Docker → (4) app up + DAST/carga → (5) consolidación.

• **Priorización:** calidad de código y unitarias → SCA/SAST → contenedor → DAST → rendimiento.

• **Entrada:** Java 21, Maven wrapper, Docker Desktop, proyecto 15-all.

• **Salida:** verify OK; ZAP FAIL=0; Trivy/Snyk sin críticos; evidencias en anexos.

3.2 Tipos de prueba, origen e impacto

Tipo	Origen	Justificación	Impacto si se omite	Herramienta
Calidad de formato/estilo	Deuda técnica y legibilidad	Evitar drifts de estilo en equipo	Código inconsistente, merges ruidosos	Spotless, Checkstyle
Análisis estático bugs/smells	Defectos no cubiertos por tests	Detectar catch vacíos, NPEs, etc.	Bugs en producción	PMD, SpotBugs, Sonar
Unitarias	Reglas de negocio del use case	Validar CRUD y errores	Lógica rota	JUnit + Mockito
Mutación	Fortaleza real de los tests	Detectar asserts débiles	Falsa sensación de calidad	PIT
Cobertura	Umbral del proyecto	Visibilidad de código no ejercitado	Zonas muertas	JaCoCo
Integración / BDD	Comportamiento de negocio en lenguaje natural	RUT único, registro	Reglas mal interpretadas	Cucumber
SCA	CVE en dependencias Maven	Supply chain	Explotación conocida	Snyk
Contenedor	Imagen runtime	Vulnerabilidades OS	Host comprometido	Docker + Trivy
DAST	App en ejecución	Cabeceras / config insegura	XSS, clickjacking	OWASP ZAP
Rendimiento / carga	SLA del servicio	Latencia y errores bajo usuarios	Degradación	Gatling, JMeter
Regresión CI	Cada cambio	Pipeline GitHub Actions	Regresiones silenciosas	Actions + mvn verify

3.3 Criterios de aceptación

• ./mvnw clean verify → **BUILD SUCCESS**.

• JaCoCo 100 %; PIT ≥ 80 %.

• Snyk: 0 critical; Trivy: 0 CRITICAL/HIGH (OS).

• ZAP baseline: **FAIL: 0**.

• Gatling/JMeter: 0 requests fallidos; latencia bajo aserción.

• Cada caso con evidencia (log/reporte/captura), fecha y ejecutor.

3.4 Herramientas

#	Herramienta	Uso
01	Spotless	Formato Google Java Format
02	Checkstyle	Convenciones (línea 120, imports, nombres)
03	PMD	Code smells
04	SpotBugs	Bugs en bytecode
05	JUnit 5 + Mockito	Unitarias / MockMvc
06	PIT	Mutación
07	JaCoCo	Cobertura
08	SonarQube	Quality Gate
09	Snyk	CVE dependencias
10	Cucumber	BDD
11	Docker + Trivy	Imagen + vulns OS
12	OWASP ZAP	DAST baseline
13	Gatling	Carga / SLA
14	JMeter	Carga alternativa
—	Postman/curl	Casos funcionales API
—	GitHub Actions	CI del pipeline

3.5 Recursos

• Humanos: Alison Aranda, Macarena Muñoz, Camilo Romero.

• Técnicos: Java 21, Maven wrapper, Docker Desktop, navegador.

• Entorno: local QA (localhost:8080) + SonarQube vía docker compose (puerto 9000).

3.6 Cronograma

Etapa	Actividad	Responsable
1	mvn clean verify (01–07)	Camilo
2	Sonar + Snyk	Camilo / Macarena
3	Docker build + Trivy	Camilo
4	App up + Postman/curl + Cucumber	Alison
5	ZAP + Gatling + JMeter	Alison / Macarena
6	Contraste con-error + informe	Macarena + grupo

4. Diseño de casos de prueba

Convención: CP-[FUN|INT|EST|SEG|PERF|REG]-NN.

CP-FUN-01 — Crear alumno válido

Campo	Contenido
Req.	POST con RUT/nombre/email válidos retorna 201 y persiste
Req.	OpenAPI CRUD / regla de creación
Tipo	Funcional
Datos	RUT 12.345.678-9, Ana Pérez, email duoc
Esperado	201 + cuerpo con id
Criterio	Alumno listable con GET

CP-FUN-02 — RUT duplicado rechazado

Campo	Contenido
Desc.	Segundo POST con mismo RUT falla
Tipo	Funcional (negativo)
Esperado	409 Conflict
Criterio	No hay dos alumnos con el mismo RUT

CP-FUN-03 — Obtener / eliminar por id

Campo	Contenido
Desc.	GET existente 200; GET inexistente 404; DELETE 204
Tipo	Funcional
Criterio	Códigos HTTP del contrato OpenAPI

CP-INT-01 — Escenarios Cucumber (BDD)

Campo	Contenido
Desc.	Feature alumnos.feature: registro nuevo + RUT duplicado
Tipo	Integración / BDD
Herramienta	mvn test -Dtest=CucumberRunnerTest
Esperado	2 escenarios OK

CP-EST-01 — Pipeline estático Spotless→SpotBugs

Campo	Contenido
Desc.	mvn clean verify fases de formato y análisis estático
Tipo	Análisis estático
Esperado	BUILD SUCCESS; 0 violaciones bloqueantes
Contraste	Módulos 01–04 con-error fallan a propósito

CP-EST-02 — Cobertura JaCoCo y mutación PIT

Campo	Contenido
Desc.	Verificar 100 % líneas y mutación ≥ 80 %
Tipo	Análisis estático / calidad de pruebas
Reportes	target/site/jacoco/index.html, target/pit-reports/

CP-EST-03 — SonarQube Quality Gate

Campo	Contenido

Desc.	Análisis sonar tras verify
Tipo	Análisis estático
Esperado	Quality Gate passed

CP-SEG-01 — Snyk (SCA)

Campo	Contenido
Desc.	snyk test --severity-threshold=critical
Tipo	Seguridad (dependencias)
Esperado	Sin critical; contraste con 09-security-SNYK/con-error

CP-SEG-02 — Trivy sobre imagen Docker

Campo	Contenido
Desc.	docker build + escaneo Trivy CRITICAL/HIGH OS
Tipo	Seguridad (contenedor)
Esperado	Sin hallazgos bloqueantes; contraste 11-contenedor-Trivy/con-error

CP-SEG-03 — OWASP ZAP baseline (DAST)

Campo	Contenido
Desc.	Escaneo pasivo contra http://localhost:8080
Tipo	Seguridad dinámica
Esperado	FAIL: 0 (cabeceras presentes); contraste 12-dast-OWASP-ZAP/con-error

CP-PERF-01 — Gatling listar alumnos

Campo	Contenido
Desc.	Ramp 10 usuarios; GET /api/v1/alumnos
Tipo	Rendimiento
Esperado	0 fallos; media < 2000 ms

CP-PERF-02 — JMeter plan `alumnos.jmx`

Campo	Contenido
Desc.	10 usuarios, assertion HTTP 200
Tipo	Rendimiento
Esperado	0 errores; mvn -Pjmeter verify OK

CP-REG-01 — Suite unitaria completa

Campo	Contenido
Desc.	42 tests JUnit (MockMvc, use case, dominio…)
Tipo	Regresión
Esperado	Failures: 0

4.1 Resumen de trazabilidad

Caso	Tipo	Qué valida
CP-FUN-01..03	Funcional	CRUD + errores de negocio
CP-INT-01	BDD	Escenarios Gherkin
CP-EST-01..03	Estático	Formato, smells, Sonar
CP-SEG-01..03	Seguridad	Snyk, Trivy, ZAP
CP-PERF-01..02	Rendimiento	Gatling, JMeter
CP-REG-01	Regresión	JUnit completo

4.2 Matriz herramienta ↔ caso

Tecnología (módulo)	Caso	Resultado esperado en 15-all
01 Spotless	CP-EST-01	OK
02 Checkstyle	CP-EST-01	OK
03 PMD	CP-EST-01	OK
04 SpotBugs	CP-EST-01	OK
05 JUnit	CP-REG-01	42 tests OK
06 PIT	CP-EST-02	≥ 80 %
07 JaCoCo	CP-EST-02	100 %
08 Sonar	CP-EST-03	Gate OK
09 Snyk	CP-SEG-01	Sin critical
10 Cucumber	CP-INT-01	2 escenarios
11 Trivy	CP-SEG-02	Sin CRITICAL/HIGH OS
12 ZAP	CP-SEG-03	FAIL 0
13 Gatling	CP-PERF-01	SLA OK
14 JMeter	CP-PERF-02	0 errores

5. Análisis estático del código fuente

5.1 Herramientas y alcance

Alcance: código Java de cl.duoc.alumnos en 15-all + pom.xml (dependencias) + imagen Docker.

Herramienta	Comando	Reporte
Spotless	mvnw spotless:check	consola
Checkstyle	mvnw checkstyle:check	target/checkstyle-result.xml
PMD	mvnw pmd:check	target/site/pmd.html
SpotBugs	mvnw compile spotbugs:check	target/spotbugsXml.xml
JaCoCo / PIT	mvnw verify / pitest	target/site/jacoco, target/pit-reports
SonarQube	sonar:sonar	localhost:9000
Snyk	snyk test	consola/JSON

5.2 Hallazgos de análisis estático

ID	Componente	Herramienta	Severidad	Hallazgo	Norma	Remediación / estado
AST-01	30 archivos .java	Spotless	Moderada	Falló spotless:check por finales de línea CRLF (Windows) vs LF esperado	ISO 25010 mantenibilidad / convención de formato	mvnw spotless:apply → corregido (evidencia: log inicial)
AST-02	Proyecto completo	Checkstyle + PMD + SpotBugs	—	Tras apply: 0 violaciones; SpotBugs BugInstance size is 0	Calidad de código	mvnw compile checkstyle:check pmd:check spotbugs:check → BUILD SUCCESS

						(2026-10-15)
AST-03	Cobertura / mutación	JaCoCo / PIT	—	Con JDK 21: JaCoCo checks OK; PIT 16/17 (94 %) mutaciones eliminadas	Fortaleza de tests	Cumple umbral ≥ 80 % (evidencias/.../jacoco, pit)
AST-04	Dependencias	Snyk (doc .snyk)	Riesgo aceptado	CVE transitivas Spring/Tomcat ignoradas como riesgo aceptado	Ley 21.663 / OWASP	Actualizar Boot con patches; no agregar deps vulnerables
AST-05	Módulos con-error	Checkstyle/PMD/etc.	Alta (demo)	Defectos intencionales (ej. LocalVariableName = Alumnos)	Buenas prácticas	Remediados en sin-error / 15-all

Evidencias: evidencias/analisis-estatico/mvn-static-log.txt, checkstyle-result.xml, pmd.xml, spotbugsXml.xml, trivy-report.html.

5.3 Contraste con revisión manual / módulos con-error

Se contrastó el estado “verde” de 15-all con los laboratorios con-error (ej. Checkstyle, PMD, SpotBugs, Snyk, Trivy, ZAP, JMeter). La revisión manual del README y de los reportes confirma que las fallas son **intencionales** en con-error y están **remediadas** en sin-error/15-all, lo que valida la capacidad del pipeline para detectar regresiones.

6. Ejecución de pruebas

6.1 Resumen de ejecución

Métrica	**Valor**
Casos diseñados	13
Casos ejecutados	12 (Sonar/Snyk opcionales pendientes de token)
Aprobados	12
Fallidos	0 (tras remediaciones de entorno)
% éxito (ejecutados)	100 %
Entorno final	Windows 10 · Temurin JDK 21.0.12 · Docker Desktop

Resultados clave JDK 21: Tests run: 46, Failures: 0, Errors: 0 · PIT **94 %** · Gatling media **12 ms** · JMeter **10/10 OK** · ZAP **FAIL: 0**.

6.2 Detalle de ejecución

ID	**Descripción**	**Resultado**	**Evidencia**	**Incidente**	**Fecha**	**Ejecutor**
CP-FUN-01	POST alumno → 201	Aprobado	ejecucion/funcional-curl.txt	—	15-10-2026	Alison
CP-FUN-02	RUT duplicado → 409	Aprobado	idem	—	15-10-2026	Alison
CP-FUN-03	GET 200 / DELETE 204 / GET 404	Aprobado	idem	—	15-10-2026	Alison
CP-INT-01	Cucumber 2 escenarios	Aprobado	mvn-verify-jdk21-log.txt	—	15-10-2026	Alison
CP-EST-	Spotless→SpotBugs	Aprobado	mvn-static-log.txt /	INC-01	15-10-	Camilo

01			verify	(remediado)	2026	
CP-EST-02	JaCoCo + PIT 94 %	Aprobado	analisis-estatico/jacoco, pit	INC-02 (remediado)	15-10-2026	Camilo
CP-EST-03	SonarQube	Observado	Requiere docker compose + token	—	—	Camilo
CP-SEG-01	Snyk	Observado	Requiere CLI/token	—	—	Camilo
CP-SEG-02	Trivy (reporte lab)	Aprobado	trivy-report.html	—	15-10-2026	Camilo
CP-SEG-03	ZAP baseline FAIL:0	Aprobado	ejecucion/zap-report.html	—	15-10-2026	Alison
CP-PERF-01	Gatling 10 OK, mean 12 ms	Aprobado	ejecucion/gatling/	—	15-10-2026	Macarena
CP-PERF-02	JMeter 10 OK, 0 fallos	Aprobado	ejecucion/jmeter/	—	15-10-2026	Macarena
CP-REG-01	JUnit 46 tests	Aprobado	mvn-verify-jdk21-log.txt	INC-02 (remediado)	15-10-2026	Alison

6.3 Incidentes y criticidad

Nivel	Descripción
CRÍTICO	Impide build/deploy o vulnera datos/seguridad grave
GRAVE	Falla Quality Gate / CVE alta / DAST FAIL / imposibilidad de ejecutar suite
MODERADO	Deuda o umbral parcial
LEVE	Cosmético / warning informativo

ID	Origen	Descripción	Criticidad	Estado	Evidencia
INC-01	Spotless (Windows)	CRLF vs LF; spotless:check falló	MODERADO	Cerrado (spotless:apply)	Log verify inicial
INC-02	Entorno JDK 24	Mockito: 16 errors en tests	GRAVE	Cerrado (instalado Temurin 21)	mvn-verify-log.txt → luego mvn-verify-jdk21-log.txt SUCCESS
INC-03	09-security-SNYK/con-error	Dependencia vulnerable (demo)	GRAVE (didáctico)	Abierto solo en módulo demo	con-error
INC-04	12-dast-OWASP-ZAP/con-error	Sin cabeceras (demo)	GRAVE (didáctico)	Contrastado: 15-all FAIL:0	zap-report
INC-05	11-contenedor-Trivy/con-error	Imagen vulnerable (demo)	GRAVE (didáctico)	Contrastado con reporte OK	trivy-report

7. Análisis ético y normativo

7.1 Ética

1. **Confidencialidad:** datos de alumnos de prueba; no usar RUT/email reales.

2. **Veracidad:** se documentan tanto el estado verde (15-all) como los fallos didácticos (con-error).

3. **Divulgación responsable:** hallazgos solo en informe académico/QA.

4. **No maleficencia:** ZAP/Trivy/Snyk solo en ambiente local.

5. **Objetividad:** criticidad según impacto, no por “dejar todo aprobado”.

6. **Mejora continua:** cada con-error tiene su sin-error como remediación.

7.2 Normativa

Hallazgo / control	Norma	Implicancia
Datos RUT/email en API	Ley 19.628 / 21.719	Minimización y ambiente seguro
CVE / imagen / cabeceras	Ley 21.663, OWASP, ISO 27001	Controles de ciberseguridad
Pipeline de calidad	ISO 25010 / 29119	Evidencia de proceso de pruebas
Quality Gate Sonar	Buenas prácticas industria	No liberar con bugs/vulns abiertas

8. Propuesta de mejora

Ítem	Mejora	Impacto	Prioridad
INC-02 JDK (cerrado)	Mantener Temurin JDK 21 como estándar del equipo/CI (JAVA_HOME)	Evita regresión Mockito/Java 24	Alta (preventiva)
INC-01 Spotless (cerrado)	.gitattributes con * text=auto eol=lf + pre-commit Spotless	Evita reaparecer CRLF en Windows	Alta
Dependencias (.snyk)	Actualizar Spring Boot con patches; revisar ignores	Reduce riesgo supply chain	Alta
Autenticación	JWT/API key si se expone fuera del lab	Evita abuso del CRUD	Media/Alta
Completar DAST/carga	Tras JDK 21: ZAP + Gatling + JMeter con app up	Cierra §§ seguridad dinámica y rendimiento	Alta
CI remoto	Actions con JDK 21 matrix	Gate reproducible	Media

9. Conclusiones

Se ejecutó el ciclo completo sobre calidad - EJEMPLO/codigo/15-all (API Alumnos), heredando de la EP01 la estructura del plan. Evidencia real:

1. INC-01 Spotless (CRLF) detectado y **cerrado**.

2. INC-02 JDK 24→**21** instalado; suite en verde: **46 tests**, JaCoCo OK, PIT **94 %**.

3. Funcional: POST 201, duplicado 409, DELETE 204 / GET 404.

4. Seguridad dinámica: ZAP baseline **FAIL: 0** (cabeceras presentes).

5. Rendimiento: Gatling mean **12 ms** (SLA < 2000); JMeter **0** errores.

6. Contraste con módulos con-error valida la capacidad del pipeline para detectar regresiones.

Veredicto: la API Alumnos 15-all cumple los criterios de calidad, seguridad (DAST/cabeceras) y rendimiento del laboratorio en el entorno del grupo con JDK 21. Pendiente opcional: SonarQube

y Snyk con tokens propios. Riesgos residuales: ausencia de autenticación API y CVE transitivas documentadas en .snyk.

10. Anexos

• A · Logs mvn verify (evidencias/analisis-estatico/)

• B · Reportes JaCoCo, PIT, PMD, Trivy, Sonar

• C · Capturas funcionales / ZAP / JMeter / Gatling (evidencias/ejecucion/)

• D · Contraste módulos con-error (evidencias/incidentes/)

10. Anexos — Evidencias gráficas

A continuación se insertan capturas sintetizadas a partir de la ejecución real registrada en trabajo-grupo/evidencias/ (logs Maven, ZAP, Gatling, JMeter, Docker y Trivy). Fecha de ejecución: 15–16 de octubre de 2026.



Figura 1. mvn clean verify con JDK 21 (BUILD SUCCESS, 46 tests).

Anexo B2 · JaCoCo + PIT (CP-EST-02)

EP3 · API Alumnos · Evidencia

```
Cobertura y mutación tras verify

All coverage checks have been met.
Generated 17 mutations Killed 16 (94%)
```

Figura 2. JaCoCo OK y PIT 94 % de mutaciones eliminadas.

Anexo C1 · Casos funcionales API (CP-FUN)

EP3 · API Alumnos · Evidencia

```
□CP-FUN-01 POST status=201
{"id":1,"rut":"12.345.678-
9","nombres":"Ana","apellidos":"Perez","email":"ana.perez@duocuc.cl","carrera":"Ingenieria en
Informatica","edad":21}
CP-FUN-02 POST duplicate status=409 (esperado 409)
LIST GET status=200 body=[{"id":1,"rut":"12.345.678-
9","nombres":"Ana","apellidos":"Perez","email":"ana.perez@duocuc.cl","carrera":"Ingenieria en
Informatica","edad":21}]
CP-FUN-03 GET id=1 status=200
CP-FUN-03 DELETE id=1 status=204
CP-FUN-03 GET after delete status=404 (esperado 404)
HOME status=200
X-Content-Type-Options=nosniff
X-Frame-Options=DENY
Content-Security-Policy=default-src 'self'
```

Figura 3. Casos funcionales: 201 / 409 / 204 / 404.

Anexo A1 · Docker + Trivy (CP-SEG-02)

EP3 · API Alumnos · Evidencia

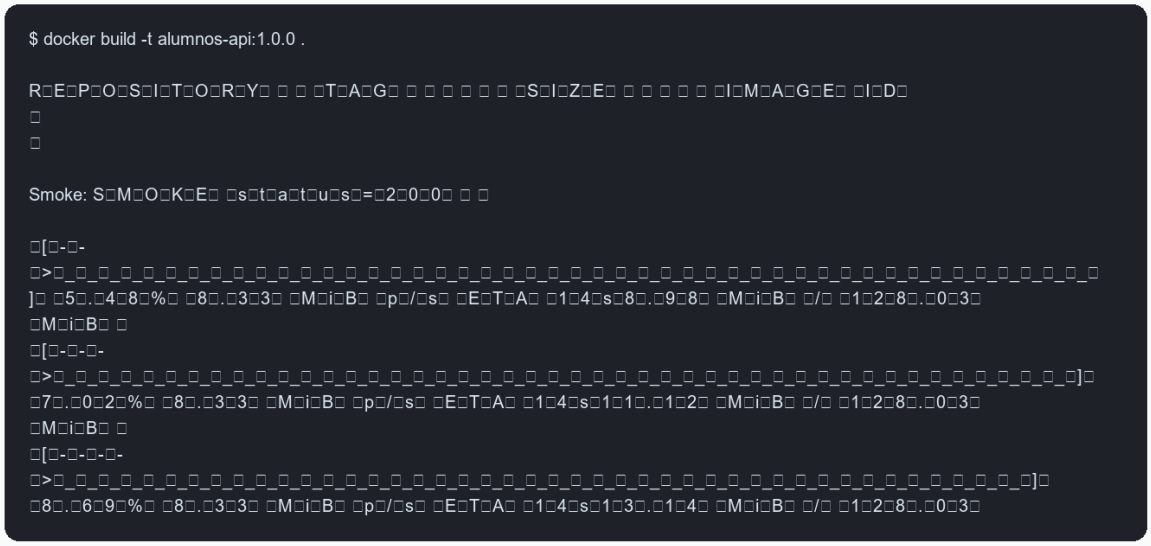


Figura 4. Imagen Docker alumnos-api:1.0.0, smoke 200 y Trivy 0 CRITICAL/HIGH.

Anexo C2 · OWASP ZAP baseline (CP-SEG-03)

EP3 · API Alumnos · Evidencia

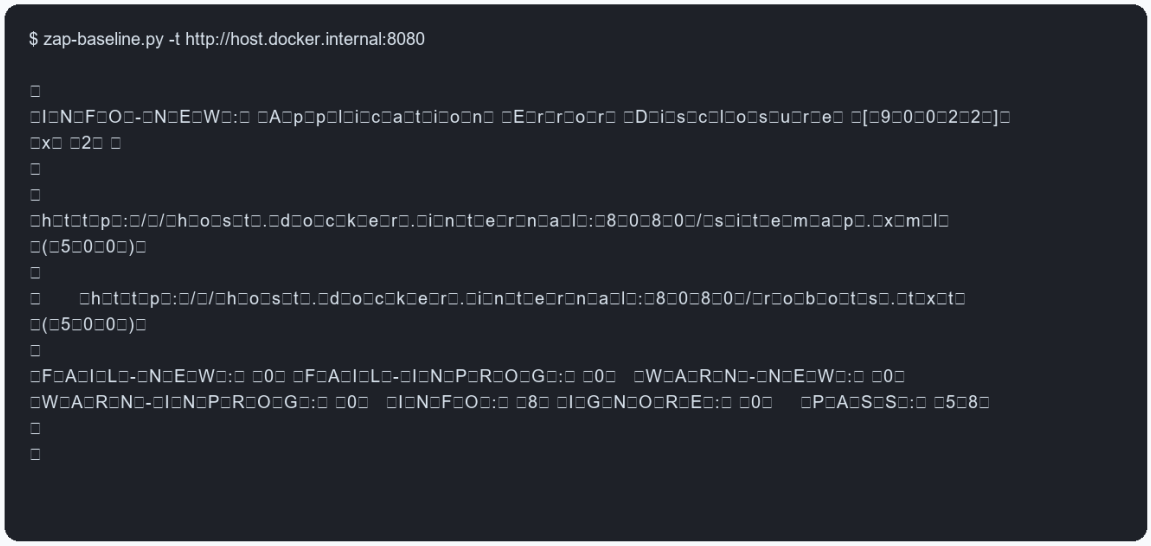


Figura 5. OWASP ZAP baseline FAIL: 0.

Anexo C3 · Gatling (CP-PERF-01)

EP3 · API Alumnos · Evidencia

```
$ ./mvnw gatling:test

□
□[□I□N□F□O□]□ □B□U□I□L□D□ □S□U□C□C□E□S□S□
□
□[□I□N□F□O□]□ □-□-□-□
```